

图算法 (Graph Algorithms) 总结 (graph I)

一、图的定义与组成 (Definitions & Components)

图是计算机科学中一种重要的数据结构，用于表示对象之间的关系。

- 组成元素：
 - 节点 (Nodes/Vertices, V): 图中的基本实体。可以是带标签或不带标签的。
 - 边 (Edges/Arcs, E): 连接节点的线。可以有向或无向的，带标签或不带标签的。
- 形式化定义：图 G 是一个有序对 (V, E) ，其中 V 是顶点集合， E 是连接顶点的边集合。
- 特殊情况：链表、树（包括二叉树、堆、B-树）都是图的特殊形式。

二、图的应用动机 (Motivation for Graphs)

图可以推广已有的数据结构，并广泛用于表示各种实际问题。

应用场景	节点 (Nodes) 表示	边 (Edges) 表示	示例解决问题
迷宫 (Maze)	房间 (Rooms)	门或通道 (Door/Passage)	寻找起点到终点的路径
电路 (Circuits)	元件 (电池、开关、电阻等)	连接关系 (Connections)	分析电流路径
程序语句	符号/操作符	关系	编译器优化 (如消除公共子表达式)
执行优先级	语句	优先级要求	确定语句的执行顺序
计算机网络	计算机	传输速率	寻找最短路径或最大流量
地图/交通	站点	连接线路	寻找最短路径

三、图的类型与术语 (Types & Terminology)

1. 有向图与无向图 (Directed vs. Undirected)

- 有向图 (Digraph): 边的顺序有意义, $(v_1, v_2) \neq (v_2, v_1)$ 。
- 无向图 (Undirected Graph): 边的顺序不重要, $(v_1, v_2) = (v_2, v_1)$ 。

2. 无向图术语

- 邻接 (Adjacent): 如果 $\{u, v\}$ 是一条边, 则顶点 u 和 v 邻接。
- 关联 (Incident): 边 $e = \{u, v\}$ 与顶点 u 和 v 关联。
- 度 (Degree, $\deg(v)$): 与顶点 v 关联的边的数量。自环 (Self-loop) 计两次。
- 握手定理 (Handshaking Theorem): 在一个有 $|E| = e$ 条边的无向图 $G = (V, E)$ 中, 所有顶点的度数之和等于边数的两倍:

$$\sum_{v \in V} \deg(v) = 2|E|$$

3. 有向图术语

- 邻接 (Adjacent):
 - 如果 (u, v) 是一条边, 则 u 邻接于 v (u is adjacent to v)。
 - v 邻接自 u (v is adjacent from u)。
- 度 (Degree):
 - 入度 (In-degree): 以该顶点为终点 (terminal vertex) 的边的数量。
 - 出度 (Out-degree): 以该顶点为始点 (initial vertex) 的边的数量。

4. 带权图 (Weighted/Labeled Graph)

- 图中的每条边可以关联一个**权值 (weight)** 或**成本 (cost)**，这样的图称为**带权图**。

5. 路径与环 (Paths and Cycles)

- **路径 (Path)**: 是一个顶点序列 $v_1, v_2, \dots, v_k$ ，使得相邻顶点间存在一条边。
 - **路径长度 (Length)**: 路径中边的数量。
 - **路径权重 (Weight)**: 路径中所有边的权重之和。
 - **简单路径 (Simple Path)**: 路径上所有顶点都不同。
- **环路 (Cycle)**: 如果 $k > 1$ 且 $v_1 = v_k$ ，则该路径是一个环路。
- **无环图 (Acyclic Graph)**: 不包含任何环路的图。
- **有向无环图 (Directed Acyclic Graph, DAG)**: 不包含任何环路的有向图。

四、图的表示方法 (Graph Representations)

图的表示方法主要有两种，其空间和时间复杂度通常用 $|V|$ (顶点数) 和 $|E|$ (边数) 来分析。

1. 邻接矩阵 (Adjacency Matrix)

- **原理**: 使用一个 $|V| \times |V|$ 的矩阵 M 来表示图。
 - $M(v, w) = 1$ (或权重) 如果边 $(v, w) \in E$ ，否则为 0。
- **空间复杂度**: $\Theta(|V|^2)$ 。
- **特点**:
 - 对于无向图，矩阵是对称的。
 - **适用于稠密图 (Dense Graphs)**，即边数 $|E|$ 接近 $|V|^2$ 的图。
 - 判断边 (v, w) 是否存在只需 $\Theta(1)$ 时间。

2. 邻接表 (Adjacency List)

- **原理**: 使用一个数组或列表，其中每个元素 $L(v)$ 存储一个链表，链表中包含所有与顶点 v 相连的邻居节点 w 。
- **空间复杂度**: $\Theta(|V| + |E|)$ (有向图) 或 $\Theta(|V| + 2|E|)$ (无向图)。
- **特点**:
 - **适用于稀疏图 (Sparse Graphs)**，即边数 $|E|$ 远小于 $|V|^2$ 的图，更节省空间。
 - 寻找顶点的邻居很高效。

五、图的实现 (Graph Implementations)

PPT中展示了图的抽象类 Graph 和两种具体的实现类 Graphm (基于邻接矩阵) 和 Graphl (基于邻接表)。

1. 抽象类 Graph 提供的基本操作

- `Init(n)`: 初始化一个包含 n 个顶点的图。
- `n()`: 返回顶点数。
- `e()`: 返回边数。
- `first(v) / next(v, w)`: 返回顶点 v 的第一个/下一个邻居。
- `setEdge(v1, v2, wgt)`: 设置或添加边 $(v1, v2)$ 及其权重。
- `delEdge(v1, v2)`: 删除边 $(v1, v2)$ 。
- `isEdge(v1, v2)`: 判断边 $(v1, v2)$ 是否存在。
- `weight(v1, v2)`: 返回边 $(v1, v2)$ 的权重。
- `getMark(v) / setMark(v, val)`: 获取/设置顶点的标记值 (常用于图遍历)。

2. 基于邻接矩阵的实现 (Graphm)

- 通过扫描矩阵行来查找邻居 (`first` 和 `next`)。

3. 基于邻接表的实现 (Graphl)

- 使用链表存储每个顶点的邻居信息 (包括邻居顶点和边权重)，更灵活地管理边。
- `first(v) / next(v, w)` 通过操作链表来访问邻居。

- 这是一个关于**图算法**，特别是**图搜索**和**图遍历**的PPT内容的总结。

lec 9-2 图算法 II：图搜索 (Graph Algorithms II: Graph Searching)

一、图遍历 (Graph Traversals)

图遍历的目的是以特定的顺序访问图的顶点，这基于图的拓扑结构。

- **核心挑战**: 必须确保在图中存在环 (Cycle) 时，算法不会进入无限循环。
- **解决方法**: 为每个顶点维护一个**标记位 (mark bit)** (例如, UNVISITED 或 VISITED)。如果遇到已标记的顶点，则不再访问第二次。
- **遍历策略**: 常见的图遍历/搜索策略有三种：
 - i. 深度优先搜索 (Depth-first search, DFS)
 - ii. 广度优先搜索 (Breadth-first search, BFS)
 - iii. 拓扑排序 (Topological sort)

二、图搜索方法论 (Graph Searching Methodology)

图搜索方法用于发现图的各种特性，如**生成树 (Spanning trees)**、**连通分量 (Connected components)**、**二分 (Bipartite) 结构**等。

1. 深度优先搜索 (DFS)

- **搜索方式**: 沿着一条路径尽可能深地搜索。
- **回溯机制**: 当一条路径上没有可用的新顶点时，算法会**回溯 (backtracks)**，并探索之前未采用的**侧路径 (side-paths)**。
- **数据结构**: 使用**栈 (Stack)**来实现 (或通过递归实现)。
- **递归算法核心**: 标记当前顶点，然后对所有相邻且未标记的顶点递归调用DFS。
- **性能**: 时间复杂度为 $O(|V| + |E|)$ ，是线性时间。

2. 广度优先搜索 (BFS)

- **搜索方式**: 使用**队列 (Queue)**来探索源顶点的邻居，然后是邻居的邻居，以此类推。
- **探索顺序**: **按距离** (边的数量) 探索顶点，即先探索所有距离为 d 的顶点，然后才探索距离为 $d + 1$ 的顶点。
- **数据结构**: 使用**队列 (Queue)**。
- **应用**: 可用于查找从起始顶点到其他顶点的**最短路径 (shortest paths)**。

三、DFS 的应用：生成树和连通分量 (Applications of DFS: Spanning Tree and Connected Components)

1. 生成树 (Spanning Tree)

- **定义**: 给定图 $G(V, E)$ ，其生成树 $G'(V', E')$ 满足 $V' = V$ (包含所有顶点)， E' 是 E 的子集，且 G' 是**连通的**且**无环 (no cycle)**。
- **DFS 的作用**: 在连通图上运行 DFS，DFS 路径上使用的边 (例如 PPT 中的红色链接) 将定义图的**生成树**。

2. 连通分量 (Connected Components)

- **定义**: 图中相互可达的顶点集合。
- **DFS 算法**: 通过 DFS 对连通分量进行**标记 (Labeling)**:
 - i. 初始化所有顶点的标签为 0。
 - ii. 从一个未标记的顶点开始，调用 DFS 并分配一个新的**连通分量标签**。
 - iii. DFS 会标记所有从起始点可达的顶点为同一个标签。

四、拓扑排序 (Topological Sort)

- **目的**: 将有向图 (通常用于表示任务依赖关系) 的顶点组织成一个**线性顺序**，使得对于任何有向边 (v, w) ，顶点 v 在排序中始终位于 w 之前 (即先完成前置任务 v 再完成 w)。
- **适用条件**: 只能应用于有向无环图 (Directed Acyclic Graph, DAG)。如果图包含循环，则无法进行拓扑排序。

1. 基于队列的算法 (Queue-based Algorithm)

该算法基于维护顶点的入度 (In-Degree)，也被称为Kahn's Algorithm。

- 1. 初始化: 计算所有顶点的入度，并将所有入度为 0 的顶点加入队列。
 - 2. 处理: 循环执行以下操作，直到队列为空:
 - 出队一个顶点并输出。
 - 将该顶点所有邻接顶点的入度减 1。
 - 如果邻接顶点的入度变为 0，则将其入队。
 - 3. 结果: 如果输出所有顶点，则排序成功；否则，图中存在环。
- 性能: 时间复杂度为 $O(|V| + |E|)$ ，是线性时间。

2. 基于 DFS 的算法

- 策略: 使用深度优先搜索策略。
- 输出顺序: 当递归回溯 (pops back)到该顶点时，打印该顶点。最终的拓扑排序是以相反的顺序打印的。

五、搜索算法总结 (Search Algorithms Summary)

算法 (Algorithm)	适用图类型 (Graph Type)	实施方式 (Implementation)	主要应用 (Key Application)
DFS	有向图和无向图 (Directed and Undirected)	栈或递归 (Stack or Recursion)	生成树、连通分量
BFS	有向图和无向图 (Directed and Undirected)	队列 (Queue)	查找最短路径 (Shortest Paths)
拓扑排序	有向无环图 (DAG Only)	DFS 或基于队列的方法 (DFS or Queue-based)	任务依赖排序

这是一个关于**最短路径算法 (Shortest-Path Algorithms)**的PPT内容总结，主要涵盖了无权图BFS算法和加权图的Dijkstra算法。

 最短路径算法 (Shortest-Path Algorithms) 总结 (graph III)

本课程 (Lecture 9-3 Graph Algorithms III) 重点介绍了图论中的最短路径问题及其主要的解决算法。

一、基本概念与问题定义

- 路径成本 (Path cost): 路径上所有边的**成本（权重）**之和。
- 路径长度 (Path length): 路径中包含的**边数**。路径长度是无权图的路径成本。
- 最短路径问题 (Single-Source Shortest Paths problem): 给定一个图 $G = (V, E)$ 和一个“源”顶点 s ，目标是找到从 s 到图中**每个顶点**的最小成本路径。
- 问题变体: 包括无权 vs. 加权、有环 vs. 无环、仅正权重 vs. 包含正负权重等。
- 应用场景: 航班安排（最低成本）、互联网数据包路由（最小延迟）、交通运输规划（最小拥堵延迟）等。

二、无权图最短路径：广度优先搜索 (BFS)

对于边没有权重的图，我们只关注**路径长度**（边数）。

- 解决方案: 使用**广度优先搜索 (BFS)**。
- 基本思想: 从源节点 s 开始，找到可以用 0, 1, 2, ... 条边到达的顶点。
- 算法概述:
 - 使用**队列 (Queue)** 跟踪“附近”的顶点。
 - 初始化源顶点 s 的距离为 0，并将其入队并标记。
 - 从队列中取出顶点 X ，遍历其所有未标记的邻居 Y 。
 - 将 Y 的距离设置为 $\text{Distance}[X] + 1$ ，并记录前驱 $\text{Previous}[Y] = X$ 。
 - 将 Y 入队并标记。

- 时间复杂度: $O(|V| + |E|)$ 。

三、加权图最短路径：Dijkstra 算法

当边具有权重时，BFS不再适用，因为最小成本路径可能比最小长度路径拥有更多的边。

3.1 Dijkstra 算法概述

- 功能: 解决**无负权重**加权图的单源最短路径问题。
- 性质: 是一种**贪心算法 (Greedy Algorithm)**。它每一步都做出当前看起来最好的选择，并且这种选择是不可撤销的。
- 基本思想:
 - i. 找到尚未“标记”的、具有最小成本的顶点。
 - ii. 标记该顶点，并更新其邻居的成本。
 - iii. 重复此过程直到所有顶点都被标记。

3.2 算法步骤

- 初始化源点 s 的成本为 0，所有其他节点的成本为 ∞ 。
- 初始化集合 S 为 $\emptyset$ (S 是已找到最短路径的节点集合)。
- 循环**直到 S 包含所有顶点:
 - 选择不在 S 中且具有最低成本的节点 A ，并将其添加到 S 中。
 - 对于 A 的每个邻居节点 B :
 - 如果 $\text{cost}(A) + \text{cost}(A, B)$ 小于 B 当前已知的成本:
 - 更新 B 的成本为 $\text{cost}(A) + \text{cost}(A, B)$ 。
 - 设置 B 的前驱节点 $\text{previous}(B) = A$ 以记录路径。

3.3 实现与复杂度分析

Dijkstra 算法的效率取决于查找未访问顶点中最小距离值的方式：

实现方法	查找最小顶点 (min <i>Vertex</i>)	边处理 (更新距离)	总时间复杂度
方法一：扫描列表	$\Theta(V)$	V	$\Theta(V^2)$, 执行
方法二：优先队列/最小堆	$\Theta(\log V)$	E	$\Theta(E \log V)$, 执行

四、限制和拓展

- 负权边: **Dijkstra 算法不适用于**包含负权边的图。
- 所有对最短路径 (All Pairs Shortest Path): 查找图中所有顶点对之间的最短路径。可以通过运行 $|V|$ 次适当的单源最短路径算法来解决。这是一个关于**最小生成树 (Minimum Spanning Tree, MST) 算法**的PPT内容总结 (Lecture 9-4 Graph Algorithms IV)。

🌳 最小生成树 (MST) 算法总结 (graph IV)

本课程介绍了在加权图中寻找连接所有顶点的最小成本树——最小生成树 (MST) 的问题和两种主要的贪心算法：**Prim 算法**和**Kruskal 算法**。

一、基本概念：生成树 (Spanning Tree)

- 定义: 给定一个连通图 $G(V, E)$ ，其生成树 $T(V', E')$ 必须满足三个条件:
 - i. **覆盖图**: 包含图中的所有顶点 ($V' = V$)。
 - ii. **无环**: 形成一个树结构 (不包含任何回路)。
 - iii. **边数**: 包含 $|V| - 1$ 条边 ($|E'| = |V| - 1$)。
- 最小生成树 (MST)**: 在边带权重的图中，拥有所有生成树中**最小总成本**的生成树。
- 应用场景**: 寻找最低成本的布线方案、确定互联网上数据传输的最小成本路径等。

二、最小生成树的通用策略（贪心法）

构建 MST 的核心策略是采用**贪心算法**：

- 选择最小边:** 每次选择一条成本最低的边。
- 避免成环:** 确保这条边与已接受的边一起**不形成环路**。
- 重复:** 重复此过程，直到选择了 $|V| - 1$ 条边。

这种策略的正确性基于一个原理：如果插入一条新边 e_{new} 形成一个环，移除环上成本更高的边 e_{old} 仍能得到一个生成树，且总成本更低。

三、两种 MST 算法

算法	构建思路	核心步骤
Prim 算法	增量式构建 (构建一棵树)	从一个随机顶点开始， 扩展已知（不完整）生成树 。在已标记顶点集 V 与未标记顶点集 U 之间，选择连接成本最小的边，并将新顶点纳入 V 中。 实现类似于 Dijkstra 算法 。
Kruskal 算法	构建森林 (最终汇集成一棵树)	按 边 的成本递增顺序选择边。如果一条边不与已接受的边形成环，则接受它。 使用并查集 (Disjoint Sets) 来检测环 。

1. Prim 算法实现要点

- 过程:** 不断选择连接**已标记节点**（当前树）到**未标记节点**（外部）中成本最小的边，直到所有节点都被标记。
- 数据结构:** 如果使用**二叉堆 (Binary Heap)** 优化“选择最小成本节点”的操作，可以获得较高的效率。

2. Kruskal 算法实现要点

- 过程:**
 - 初始化一个森林，每个节点都是一棵树。
 - 将所有边放入一个**优先队列**（按成本排序）。
 - 取出最小成本边 (u, v) ，如果 u 和 v 属于不同的树（ $\text{Find}(u) \neq \text{Find}(v)$ ），则接受该边，并使用 **Union** 操作合并两棵树。
 - 如果 u 和 v 属于同一棵树，则加入边会形成环，丢弃该边。

四、时间复杂度总结

假设 $|V| = n$ (顶点数), $|E| = m$ (边数), 且 $m = O(n^2)$:

算法	实现方法	渐进时间复杂度
Prim's	使用二叉堆优化	$O((n + m) \log n)$
Kruskal's	使用优先队列和并查集	$O(m \log m)$ 或 $O(m \log n)$
实际表现		在 稀疏图 中，Kruskal 算法由于无需查看所有边，通常运行得更快。